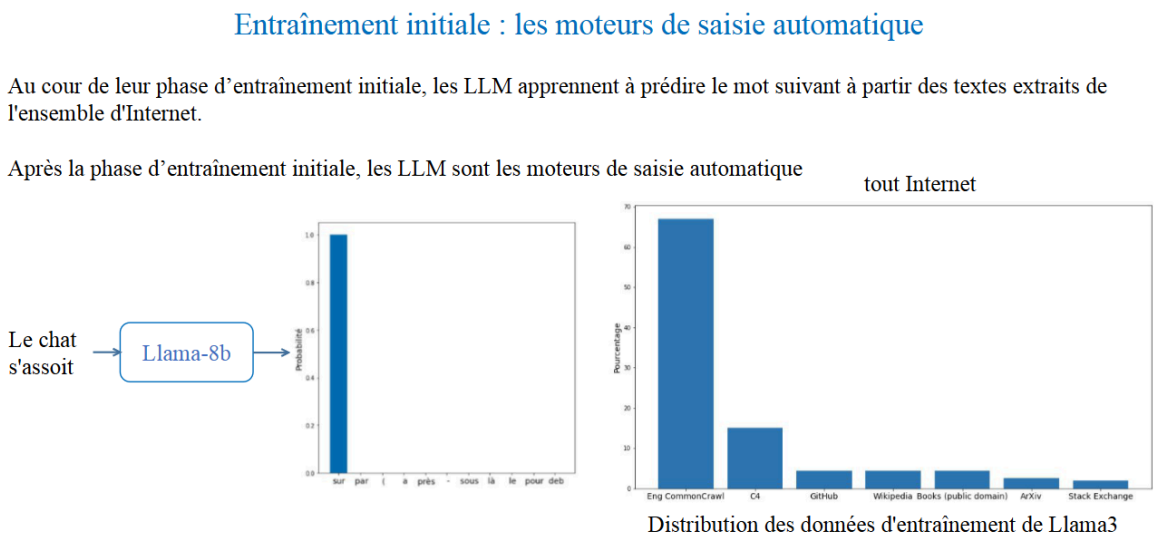
Concepts et usages de l'IA gen
file:///C:/Users/durandl/Downloads/concept_usage_IA_part1.pdf



1. Fonctionnement LLM

1.1. Composants essentiels

LLM = Large Language Model = une fonction qui transforme une chaîne de caractères en entrée en une distribution de probabilité du mot suivant.

RAG = technologie qui consiste à améliorer les réponses des modèles d'IAG en les alimentant avec des connaissances issues des DB internes de l'entreprise

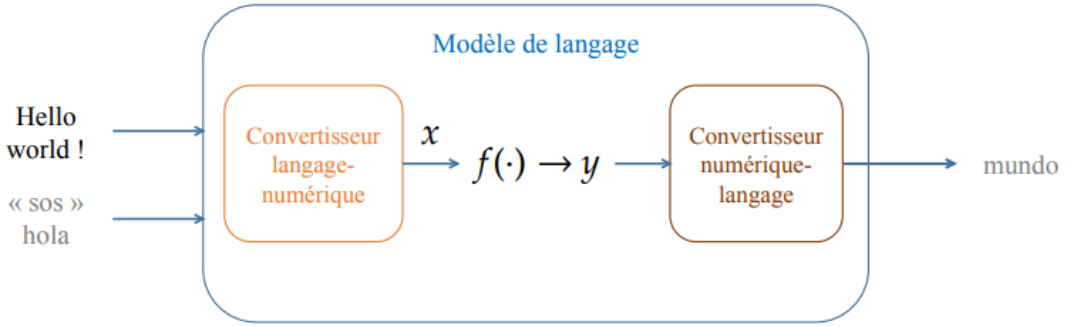
1.1.1. Modèle de langage

génération autorégressive = génère du texte mot par mot en se basant sur les précédents mots produits. À chaque étape, il prédit le mot le plus probable qui suit la séquence déjà générée.

Unicode comme ASCI mais plus grand

Tokenisation

Le modèle ne comprend pas les mots.
Il comprend des tokens (morceaux de mots) convertis en entiers.
Donc découpe en jetons (nombre différents selon les grandes IA)



+ B.o.n.j.o.u.r → | Unicode | → 100,121,252 .. → | UTF-8 |

Ex : *"Hello world !"*
→ *normalisation : hello world*
→ *tokenisation → ['sos', 'hello', 'world', 'eos']*
→ *encodage → [1, 1778, 321, 2]*
Ce sont les étapes du Convertisseur langage-num

Embeddings

Convertir du texte en vecteurs, donc en listes de nombres, en fonction de caractéristiques.
Besoin de multidimensionnel parce que le sens d'un mot n'est pas simple.
Chaque dimension encode un petit morceau d'info : connotation, contexte, proximité grammaticale, domaine, etc.

🧪 Exemple concret

Imagine un espace 3D :

- axe 1 : animal ↔ objet
- axe 2 : domestique ↔ sauvage
- axe 3 : petit ↔ grand

Tu peux déjà placer pas mal d'animaux.

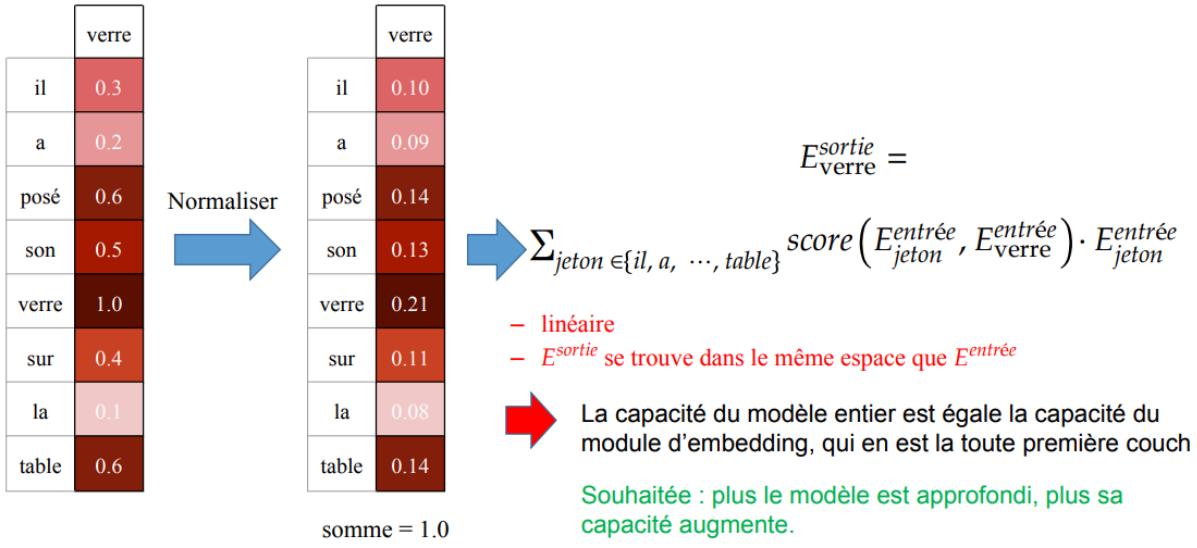
Ex :

- *255 → V_{255} ∈ R**n avec n la dimension embeddings, ex 4018*
- *matrice de taille MxN avec M la taille du vocabulaire*
- *$V^*_{100} = f(V_{100}, V_{120}, V_{258}) = W_{100}V_{100} + W_{120}V_{120} + \ldots$*
  use similarité entre V_{100} et V_{120} :
  = cos((V_{100}, V_{120})) = V_{100} produit scalaire V_{120} = $||V120|| \cdot ||V_{100}|| \in [0,1]$

Self-attention

Important de prendre en compte le contexte, il est mis sous forme de score d'attention.

- "Plus le modèle est approfondi, plus sa capacité augmente."
- Calcul complexe avec produit scalaire et FNN
- Complexité O(N**2) donc taille dépend nb de calculs ex 1000tokens = 1M calcul et 10000tokens = 100M de calculs (voir slide1 p59 pour exemples IA connus)



(optionnel)

Pour chaque mot, le modèle crée trois vecteurs :

- **Q** = *Query* → "ce que je cherche"
- **K** = *Key* → "ce que je représente"
- **V** = *Value* → "l'information que j'apporte"

Puis il fait ça :

1. Il compare **Q** d'un mot avec **K** de tous les autres mots.
   → Ça sort des scores : plus c'est haut, plus ce mot est important pour moi.
2. Il transforme ces scores en **probabilités** (softmax).
   → Ça devient des poids d'attention.
3. Il combine ces poids avec les **V**.
   → Ça donne une *nouvelle représentation* du mot, enrichie du contexte.

Cross-attention = fusionne contexte d'entrée et de sortie

FNN (Feedforward Neural Network)

+ Sim(V_{100}, V_{120}) → $FNN(x) = y$
+ $V_{100} = FNN(V_{120})$
+ $y = RLU(W*x+b)$

Le FNN (Réseau de Neurones à Propagation Avant) est un composant fondamental des architectures de modèles d'IA générative (comme les Transformeurs utilisés dans les LLM).

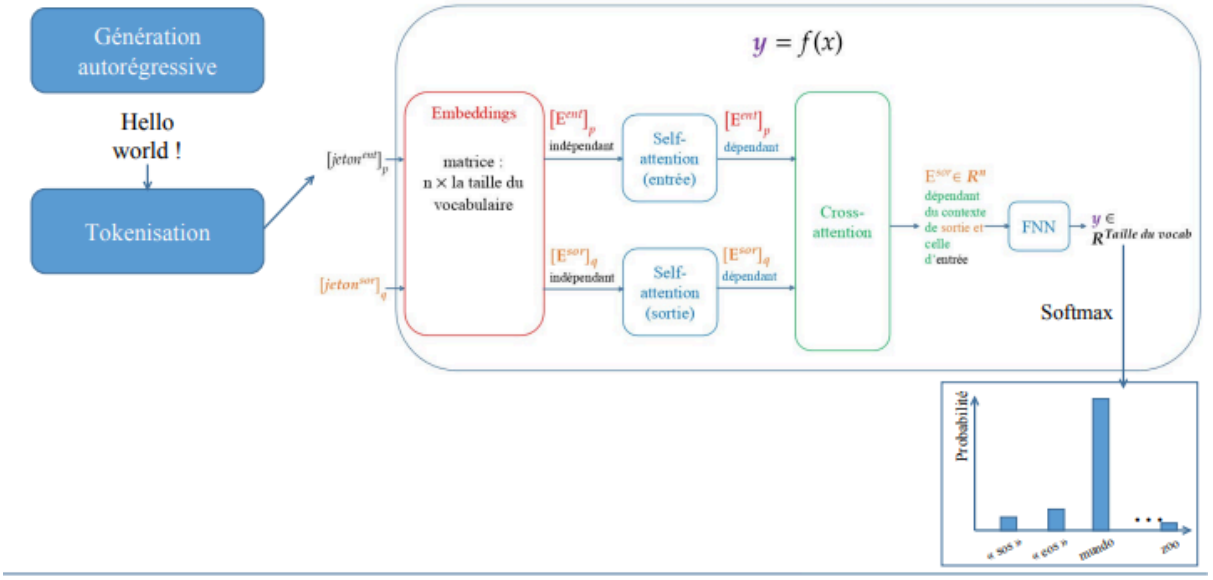
Il s'agit d'un réseau neuronal simple où l'information se déplace uniquement vers l'avant, de la couche d'entrée à la couche de sortie, sans boucle.

Dans le contexte des LLM :

- **Rôle :** Après le mécanisme d'auto-attention (Self-attention), le FNN est appliqué indépendamment à chaque position de *token* pour transformer et projeter le vecteur d'attention dans un espace de représentation plus riche.
- **Fonctionnement :** Il se compose généralement de deux couches linéaires avec une fonction d'activation non linéaire (comme **RLU** ou ReLU, mentionnée dans la section *Cross-attention*) entre elles.
- **Objectif :** Il permet d'introduire de la non-linéarité et de transformer les représentations apprises par l'attention, augmentant ainsi la capacité du modèle à capturer des relations complexes dans les données.

1.1.2. Choisir mot suivant

Jetons d'entrées → vecteur → calcul via exp et division → probas → jetons de sortie
Echantillonnage glouton : i=argmax{y0,y1…,ytaille_voc} cad rend la plus grande proba cumulative



1.2. Entrainements

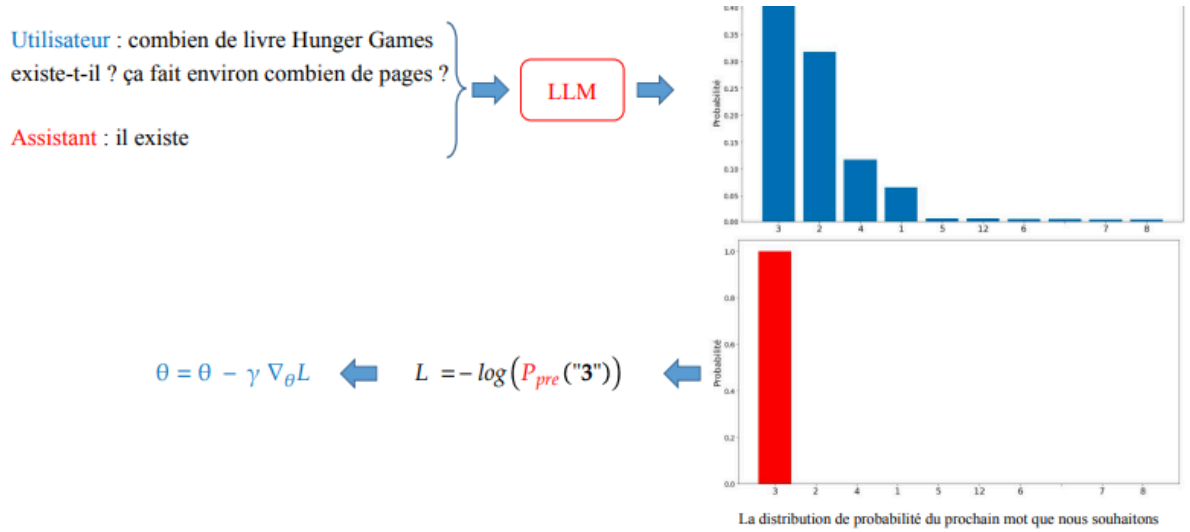
Entrainement = ajuster poids du modèle afin de minimiser l'écart entre la prédiction et le résultat attendu

- Données proviennent de tout internet mais voles
- Enormes consommations en GPU et eau pour refroidissement

Comportement des LLM après : Complétion de texte + Mémoriser textes de son base d'entrainement + conaissances limitées

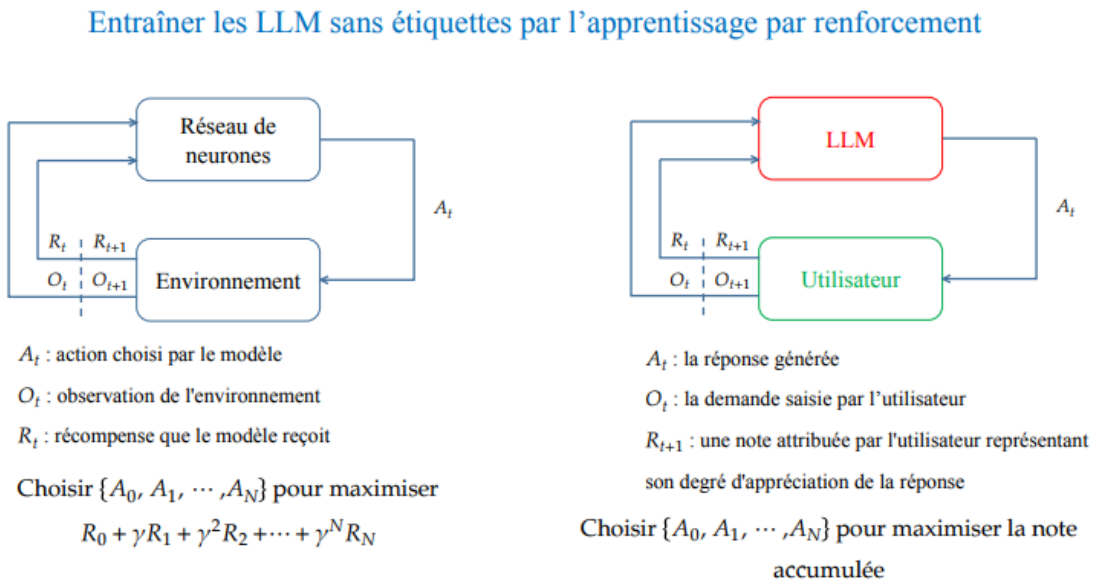
1.2.1. Transfo moteur de saisie en assistant

Pour qu'il se comporte comme tel, on doit lui montrer les rep attendues
Affiner avec des données de conv

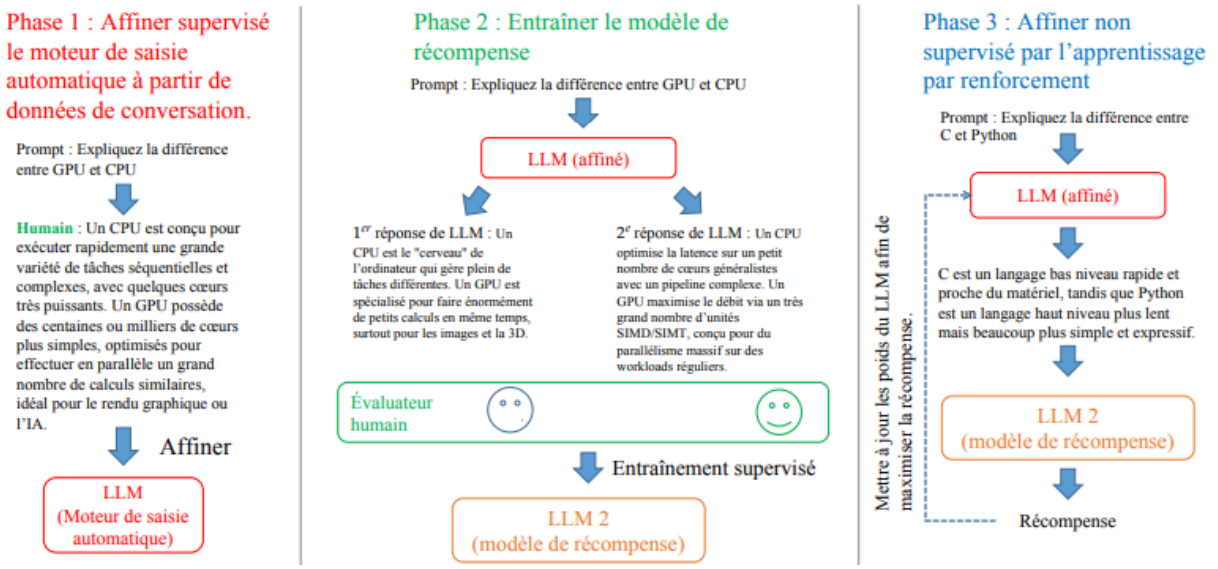


Pb : Créer des convs pour entraîner est laborieux donc use 2ème LLM pour simu préferences humaines

1.2.2. Apprentissage par renforcement



RLHF (Reinforcement Learning from Human Feedback) : Méthode clé pour aligner le LLM sur les préférences humaines : (+ voir ex HTML)



Phase 1 : SFT (Supervised Fine-Tuning) :
Affiner le modèle initial sur un jeu de données de haute qualité (paires instruction-réponse).

Phase 2 : Modèle de récompense (RM) :
Entraîner un modèle pour prédire la qualité/préférence humaine pour une réponse donnée.

- Collecte de données de préférence (comparaisons de réponses).
- RM apprend à attribuer un score de récompense.

Phase 3 : Optimisation RL :
Utiliser le RM comme fonction de récompense pour affiner le LLM via un algorithme d'apprentissage par renforcement (ex : PPO).

- Objectif : Maximiser la récompense tout en maintenant la proximité avec le modèle SFT (terme de régularisation KL).
- Résultat : LLM plus utile, honnête et inoffensif (alignment).

“Phase 1 = moteur de saisie auto + Phase 2 = transfo en assistant + “

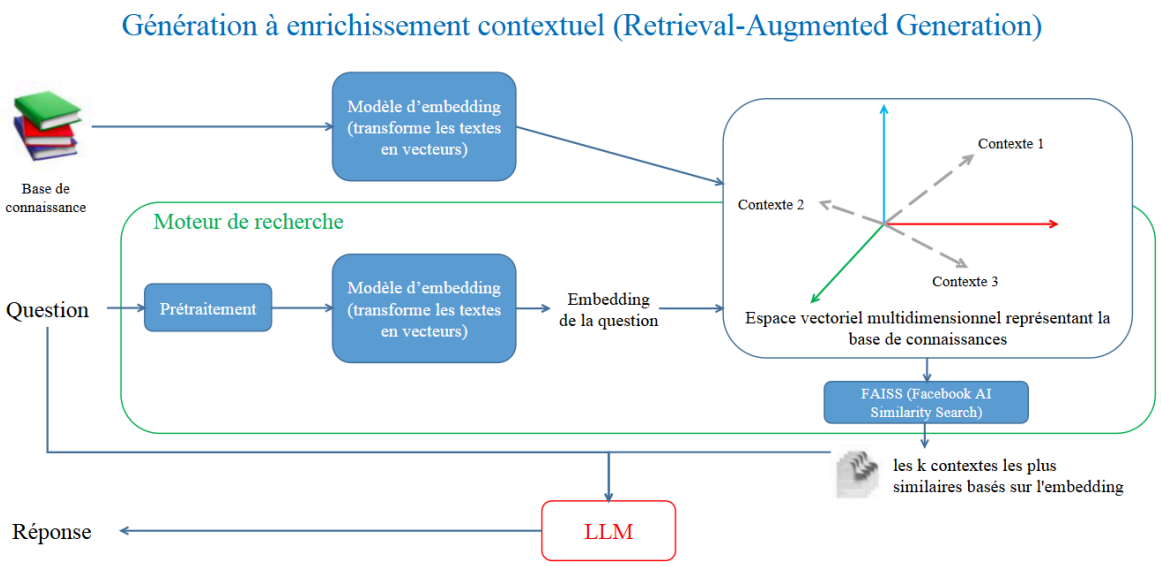
2. Les limites

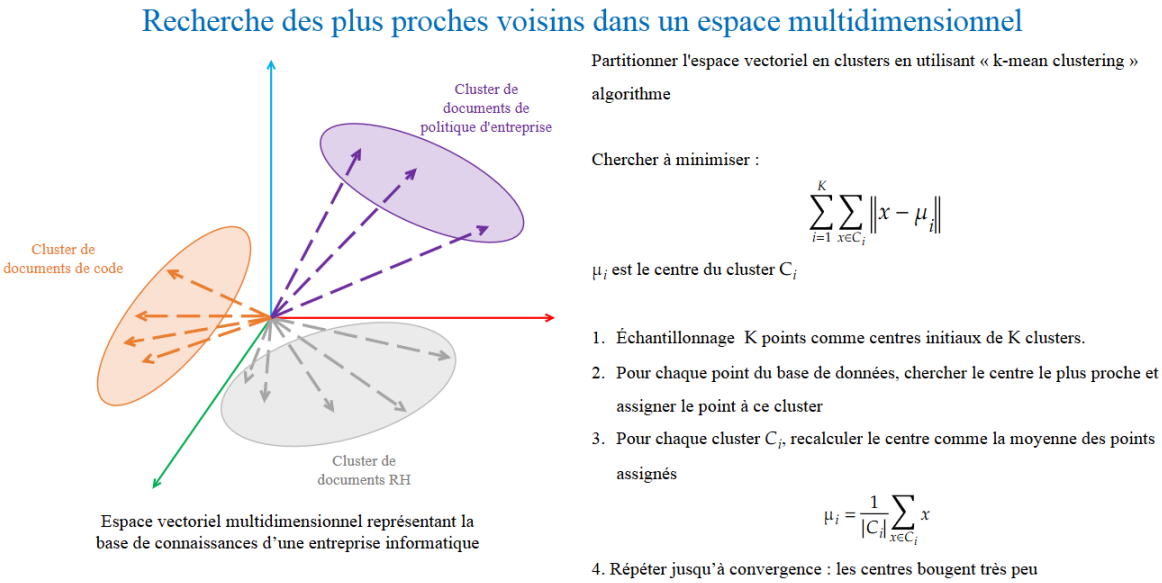
2.1. Coupures de conaissances

- Entrainement long
- Informer du contexte

RAG

Entraîner avec DB spécial pour améliorer la qualité des réponses





Rechercher le cluster des documents dont le centre est le plus proche de l'embedding de la requête

Parmi tous les documents de ce cluster, rechercher les k documents dont l'embedding est le plus proche de l'embedding de la requête.

2.2. Hallucination

Les LLM ne peuvent pas mémoriser parfaitement ses données d'entraînement

Se produit lorsque les LLM génèrent des réponses plausibles, mais factuellement incorrectes.

Nous ne pouvons pas savoir avec certitude

- quels sont les éléments dont LLM ne se souvient pas

0,1 % des données d'entraînement par des informations erronées peut entraîner une augmentation de 20 % de la fréquence des réponses dommageables.

Solution → RAG ou faire dire “jsp” :

1. Poser des questions aux modèles pour étudier ce qu'ils ne savent pas
2. Créer une base de données contenant des faits que les modèles ne connaissent pas
3. Affiner le modèle pour dire « je ne sais pas » à l'aide de cette base de données

3. Raisonnement

Raisonnement : LLM génèrent davantage de jetons intermédiaires avant d'arriver à la réponse finale

Find the degree measure of the least positive angle θ for which

$$\tan\theta = \frac{\cos 5^\circ \cos 20^\circ + \cos 35^\circ \cos 50^\circ - \sin 5^\circ \sin 20^\circ - \sin 35^\circ \sin 50^\circ}{\sin 5^\circ \cos 20^\circ - \sin 35^\circ \cos 50^\circ + \cos 5^\circ \sin 20^\circ - \cos 35^\circ \sin 50^\circ}.$$

LLM (assistant)

I notice that the numerator and denominator of the fraction are both sums of products of cosine and sine functions of angles that add up to 25 degrees or 85 degrees.

This suggests that I might be able to use some trigonometric identities to simplify the expression.

For example, I recall that $\cos(A+B) = \cos A \cos B - \sin A \sin B$ and $\sin(A+B) = \sin A \cos B + \cos A \sin B$.

If I apply these identities to the terms in the numerator, I get $\cos 5^\circ \cos 20^\circ + \cos 35^\circ \cos 50^\circ - \sin 5^\circ \sin 20^\circ - \sin 35^\circ \sin 50^\circ = \cos(5^\circ + 20^\circ) + \cos(35^\circ + 50^\circ) = \cos 25^\circ + \cos 85^\circ$.

Similarly, if I apply these identities to the terms in the denominator, I get $\sin 5^\circ \cos 20^\circ - \sin 35^\circ \cos 50^\circ + \cos 5^\circ \sin 20^\circ - \cos 35^\circ \sin 50^\circ = \sin(5^\circ + 20^\circ) - \sin(35^\circ + 50^\circ) = \sin 25^\circ - \sin 85^\circ$.

Chain-of-thought = Chaîne de pensée Leur montrer comment raisonner
(Plus les modèles sont grands, plus les perfs augmentent)

- Générer automatiquement des chaînes de pensées en utilisant les modèles de récompenses (à entraîner eux aussi).